

“Club Attendance System”

*Project report submitted
in
Partial fulfilment of the requirement for the degree of*

Masters in Computer Applications (AI & ML)

Semester 1 / Semester - Examination

Submitted by

Miss Purvi Prakash Dabhade.

Roll no. : 43

Section / Batch : A / A2

Under the guidance of

Prof. Achal Rokade Mam

Asst. Professor, Department of Computer Science and Applications,
RBU, Nagpur.



**Department of Computer Science & Applications,
School of Computer Science and Engineering,
RAMDEOBABA UNIVERSITY, NAGPUR.**

2025-2026

Table of Contents

Sr. No.	Title	Page No.
1.	Introduction	04
2.	Aim and Objectives	06
3.	Software & Hardware Requirements	07
4.	System Design/ Project Flow	08
5.	Java Code Implementation	11
6.	Output Screenshots	15
7.	Result/Testing	22
8.	Conclusion	25
9.	Future Scope	26
10.	References	27

List of Images

Sr. No.	Figure/Image Title	Page No.
1.	Figure1 : System Flowchart	10
2.	Figure2 : Java Concepts Used	11
3.	Image1 : Main (Login) Page	15
4.	Image2 : Head Login Page	15
5.	Image3 : Coordinator Login Page	16
6.	Image4 : Member Login Page	16
7.	Image5 : Head Dashboard Page	17
8.	Image6 : Coordinator Dashboard Page	17
9.	Image7 : Member Dashboard Page	18
10.	Image8 : Member Registration Page	18
11.	Image9 : Meeting Code Creation Page	19
12.	Image10 : Attendance Marking Page	19
13.	Image11 : Attendance Sheet View Page	20
14.	Image12 : Showing Denial of Login for Invalid Credentials	20
15.	Image13 : Showcasing how member can mark attendance.	21
16.	Image14 : Showing attendance is marked only for correct meeting code.	21
17.	Image15 : Showing for incorrect meeting code, Attendance is not marked.	22

INTRODUCTION

The **Club Attendance System** is a Java-based web application designed to simplify, automate, and secure the process of tracking attendance within a club or organizational environment. Traditional attendance methods—such as manual registers or spreadsheets—tend to be time-consuming, error-prone, and inefficient, especially when managing large groups. This project overcomes these limitations by providing a structured digital platform where all attendance-related activities can be managed seamlessly.

This system offers three distinct dashboards: **Head**, **Coordinator**, and **Member**, each with its own access privileges and functionalities. The Head is responsible for managing meetings, generating meeting codes, adding members, marking attendance, and generating reports. Coordinators can create meeting codes, mark attendance, and view attendance records, while Members can log in, mark their own attendance, and view their attendance history.

The application is developed using **JSP, HTML, CSS, JavaScript**, and **Java file handling**, which is used to store login credentials, meeting codes, and member information without relying on a database. The use of file handling ensures simple, fast, and reliable data management suitable for lightweight applications.

Overall, the project aims to enhance accuracy, transparency, and efficiency in attendance tracking, while offering an easy-to-use interface accessible to all club roles. Managing club attendance manually is time-consuming, error-prone, and often leads to missing or duplicated records. To overcome these limitations, this project introduces a **web-based attendance tracking system** with role-based login access. The system ensures structured management of members, meetings, and attendance through a simplified interface and automated storage.

The project demonstrates the use of **JSP for backend processing**, **HTML/CSS for UI**, **JavaScript for validation and dynamic features**, and **Java File Handling** for persistent data storage.

Overview:

JAVA CONCEPTS USED

- **Object-Oriented Programming (OOP) :**
Encapsulation, Inheritance, and Polymorphism are used to design and implement the system's classes and objects.
- **Java Servlet Pages (JSP) :**
Used for creating dynamic web pages and handling user interactions.
- **File Handling :**
Used for storing and retrieving data, such as member information and attendance records.

SYSTEM FUNCTIONALITIES

- **Club Head Login :**
 - Member registration (add member)
 - Create meeting code
 - Mark you own attendance
 - Attendance sheet (downloadable)
- **Coordinator Login :**
 - Create meeting code
 - Mark your own attendance
 - Attendance sheet (downloadable)
- **Member Login :**
 - Mark attendance
 - Read meeting attendance

SYSTEM DESIGN

- **Login Servlet :** Handles user authentication and redirects to the respective dashboard.
- **Member Registration Servlet :** Handles member registration and stores data in the file.
- **Meeting Code Servlet :** Generates and stores meeting codes.
- **Mark Attendance Servlet :** Stores the list of members who have marked their attendance.
- **Attendance Sheet Servlet :** Handles attendance tracking and generates attendance sheet.

FILE HANDLING DESIGN

The file handling will consist of the following tables :

- **Member :**
 - Stores member information (name, branch, year, password).
- **Meeting codes :**
 - Stores meeting information (meeting code, date).
- **Attendance :**
 - Stores attendance records (member's username, member's branch, member's year, meeting code, attendance status).

AIM & OBJECTIVES

AIM :

The aim of the Club Attendance System Project is to design and develop an efficient, user-friendly, and automated system for managing club attendance for every club meeting happening in club, enhancing member engagement and reducing administrative tasks.

OBJECTIVES :

- 1. To provide role-based authentication**
 - Separate login modules for **Head**, **Coordinator**, and **Member** with secure access using credentials.
- 2. To allow the Head to manage club operations**
 - Register new members by storing their details in a text file.
 - Create and generate unique meeting codes with date and time.
- 3. To enable members to mark attendance digitally**
 - Members can log in using registered credentials and mark attendance only with a valid meeting code.
- 4. To validate attendance using meeting codes**
 - Attendance is marked only when the entered meeting code matches the most recently generated code, preventing fake or duplicate entries.
- 5. To store attendance records using file handling**
 - Attendance details such as name, branch, year, meeting code, date, and status are stored in text files for simplicity and transparency.
- 6. To display an organized attendance sheet**
 - Show attendance records in a tabular format with meeting details like latest meeting code and date.
- 7. To support downloading attendance records**
 - Allow the Head and Coordinator to download attendance reports in **TXT and PDF (print)** format.
- 8. To ensure session-based security**
 - Prevent unauthorized access to dashboards using session validation and proper redirection.
- 9. To design a user-friendly interface**
 - Implement responsive and attractive UI using **CSS** for better usability and readability.
- 10. To implement a lightweight system without a database**
 - Use text files instead of databases to make the system easy to understand, deploy, and suitable for academic projects.

SOFTWARE AND HARDWARE

Software Required:

- Java (JDK 8 or above)
- Visual Studio Code
- Web Technologies : Java Server Pages (JSP), HTML5, CSS3, JavaScript (for validation and downloads)
- Operating System : Windows10/ Windows11/ Linux
- Web Browser : Google Chrome / Mozilla Firefox / Microsoft Edge
- Web Server : Apache Tomcat Server
- JSP and Servlets
- Data Storage/File System Storage for storing TXT files

Hardware Required:

- Any desktop/laptop with minimum 4GB RAM
- Processor: Intel i3 or above
- RAM: 4GB minimum
- Storage: 500MB free
- Input Devices like keyboard, mouse. (Optional)

SYSTEM DESIGN / PROJECT FLOW

The **Club Attendance System** follows a simple, role-based workflow using JSP, sessions, and file handling. The system is divided into logical modules to ensure secure access and smooth operation.

Overall Project Flow:

1. User opens the Club Attendance System.
2. User selects the appropriate login role:
 - Head
 - Coordinator
 - Member
3. System authenticates the user based on stored credentials.
4. After successful login, the user is redirected to their respective dashboard.
5. Each role performs specific operations as per permissions.
6. All data is stored and retrieved from text files.
7. Sessions are used to maintain secure login states.
8. User logs out and session is destroyed.

Head Module Flow:

1. Head logs in using Head credentials.
2. Upon successful authentication:
3. Head Dashboard is displayed.
4. Head can:
 - Register new members
Member details (Name, Branch, Year, Password) are saved in member.txt.
 - Create meeting codes
Meeting code along with date is saved in meetingcodes.txt.
5. Head can view attendance sheet.
6. Head can download attendance reports (TXT / PDF).

Coordinator Module Flow:

1. Coordinator logs in using Coordinator credentials.
2. Coordinator Dashboard is displayed.
3. Coordinator can:
 - View attendance records.
 - Download attendance reports.
4. Coordinator has read only access but can create meeting code.

Member Module Flow:

1. Member logs in using credentials stored in member.txt.
2. System validates name and password.
3. After successful login:
4. Member Dashboard is shown.
5. Member enters:
 - Name
 - Branch
 - Year
 - Meeting Code
6. System checks:
 - Whether the entered meeting code matches the most recently generated meeting code.
7. If valid:
 - Attendance is marked as Present.
 - Entry is stored in attendance.txt.
8. If invalid:
Attendance is rejected with an error message.

Data Flow Design:

File Name	Purpose
Member.txt	Stores registered member details
Meetingcodes.txt	Stores generated meeting codes with date
Attendance.txt	Stores attendance marked by users

Session Management Flow:

1. Session is created after successful login.
2. Session attributes identify:
 - Role (HEAD / COORDINATOR / MEMBER)
 - User name and details
3. Each dashboard checks session attributes.
4. Unauthorized access redirects user to login page.
5. Logout destroys the session.

Security Flow:

- Role-based authentication.
- Session validation on every dashboard.
- Attendance allowed only for current meeting code.
- No direct dashboard access without login.

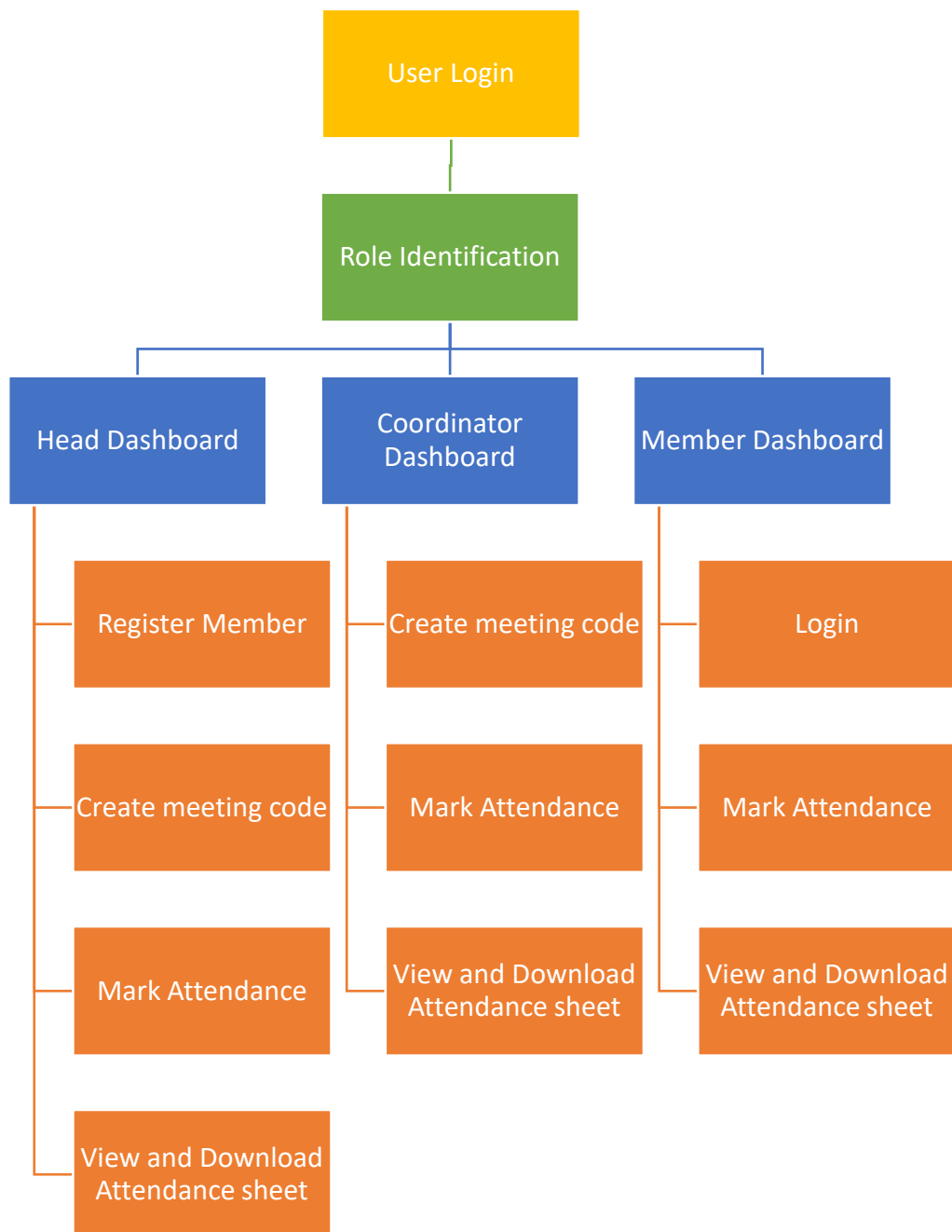
System Flowchart:

Figure1 : System Flowchart

JAVA CODE IMPLEMENTATION

Java Concepts Used:



Figure2 : Java Concepts Used.

1. Class

- **Theory :** A class is a blueprint that represents a logical entity. It defines behaviour and structure of objects.
- **Application in Project :** In the project, each JSP file acts as a class. Internally, the JSP engine converts each JSP page into a Java servlet class.
- **Code Example –**

```

<%@ page import="java.io.*" %>
<%
String u = request.getParameter("user");
String p = request.getParameter("pass");
%>
  
```

- **Explanation :** Here, HeadLogin.jsp works as a class that handles Head login logic

2. Object

- **Theory :** An object is an instance of a class created at runtime using the new keyword.
- **Application in Project :** Objects of Java I/O classes are created to handle file operations.
- **Code Example :**

```
File file = new File(application.getRealPath("data/member.txt"));
BufferedReader br = new BufferedReader(new FileReader(file));
```

- **Explanation :**

file is an object of the File class
br is an object of the BufferedReader class

3. Encapsulation

- **Theory :** Encapsulation is the process of wrapping data and methods together and restricting direct access to data.
- **Application in Project :** User details and role information are stored inside **session objects**.
- **Code Example :**

```
session.setAttribute("role", "HEAD");
Checking encapsulated data:
if (session.getAttribute("role") == null ||
    !"HEAD".equals(session.getAttribute("role"))) {
    response.sendRedirect("HeadLogin.jsp");
}
```

- **Explanation :** The role value is protected and can be accessed only through session methods.

4. Abstraction

- **Theory :** Abstraction hides complex implementation details and shows only essential functionalities.
- **Application in Project :** JSP and Servlet APIs provide abstraction of HTTP request handling.
- **Code Example :**

```
String name = request.getParameter("name");
response.sendRedirect("MemberDashboard.jsp");
```

- **Explanation :** The programmer does not handle low-level HTTP processing. It is managed internally by the server.

5. Inheritance

- **Theory :** Inheritance allows a class to acquire properties and methods of another class.
- **Application in Project :** Every JSP page is converted into a servlet that inherits from HttpServlet.
- **Conceptual Code Example :**

```
public class MemberLogin_jsp extends HttpServlet {  
}
```

- **Explanation :** The JSP servlet inherits request-handling methods such as doGet() and doPost().

6. Polymorphism

a. Runtime Polymorphism

- **Theory :** Runtime polymorphism is achieved through method overriding, where the method behavior is decided at runtime.
- **Application in Project :** Different JSP pages override doPost() internally.
- **Code Example :**

```
if ("POST".equalsIgnoreCase(request.getMethod())) {  
    // Different logic in different JSP pages  
}
```

- **Explanation :** Same method, different behavior depending on the request.

b. Compile-Time Polymorphism

- **Theory :** Compile-time polymorphism occurs through method overloading.
- **Note :** This project does not explicitly use method overloading as it focuses on runtime web request handling.

7. File Handling

- **Theory :** File handling allows data to be stored permanently in files.
- **Application in Project :** Text files are used instead of a database.
- **Code Example (Writing Data) :**

```
FileWriter fw = new FileWriter(  
    application.getRealPath("data/attendance.txt"), true);
```

```
fw.write(name + " | " + branch + " | " + year + " | " + code + " | Present\n");  
fw.close();
```

- **Code Example (Reading Data) :**

```
BufferedReader br = new BufferedReader(  
    new FileReader(application.getRealPath("data/attendance.txt")));  
String line;  
while ((line = br.readLine()) != null) {  
    // process record  
}
```

- **Explanation :** This ensures persistent storage of attendance records.

8. Session Management

- **Theory :** Session management maintains user state across multiple pages.
- **Application in Project :** Used for:
 - Secure login
 - Role-based access
 - Logout
- **Code Example :**

```
session.invalidate();  
response.sendRedirect("login.jsp");
```

- **Explanation :** Session termination ensures secure logout.

9. Conclusion

The Club Attendance System effectively demonstrates **Java Object-Oriented Programming concepts** using JSP. The implementation of classes, objects, encapsulation, abstraction, inheritance, and polymorphism provides modularity and security. File handling enables persistent data storage, while session management ensures secure role-based access. This project highlights the practical application of Java OOP principles in developing a real-world web-based system.

OUTPUT SCREENSHOTS

Image1: Main (Login) Page

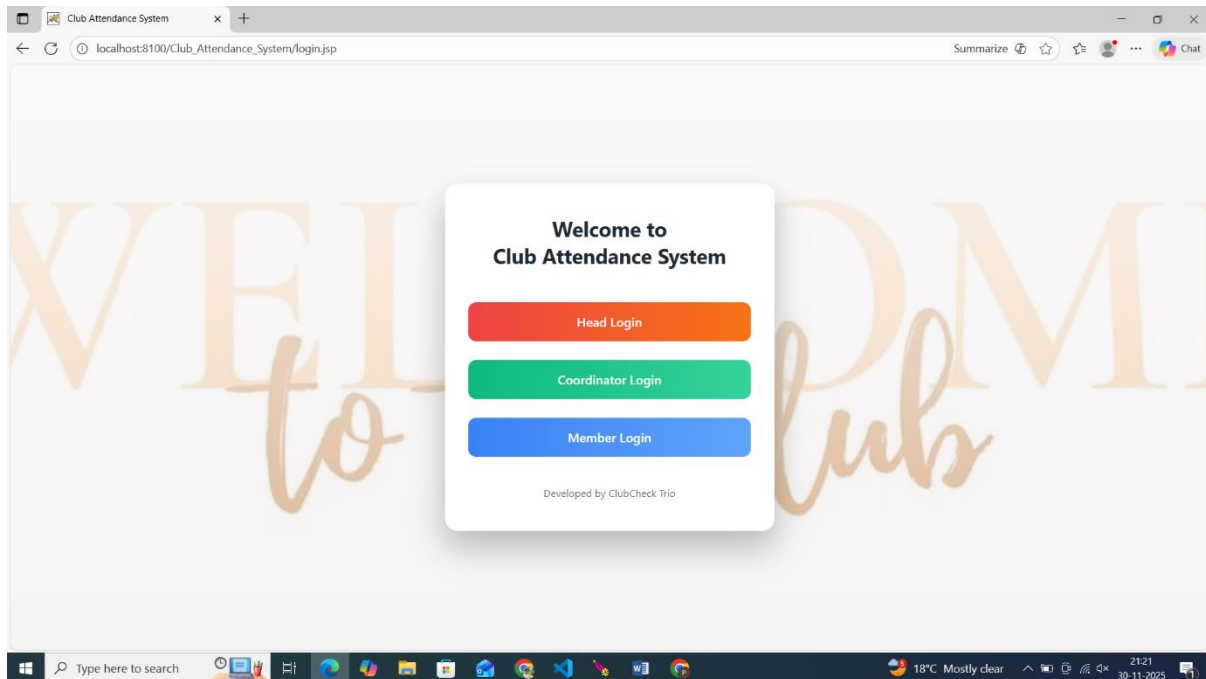


Image2 : Head Login Page

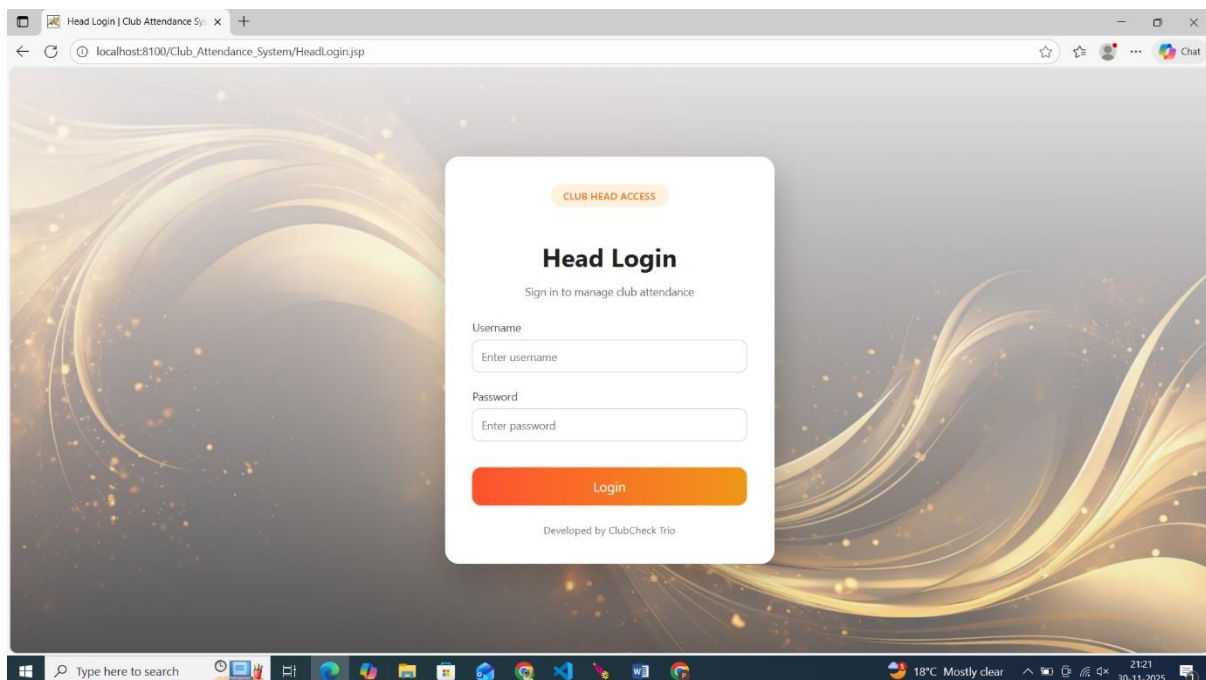


Image3 : Coordinator Login Page

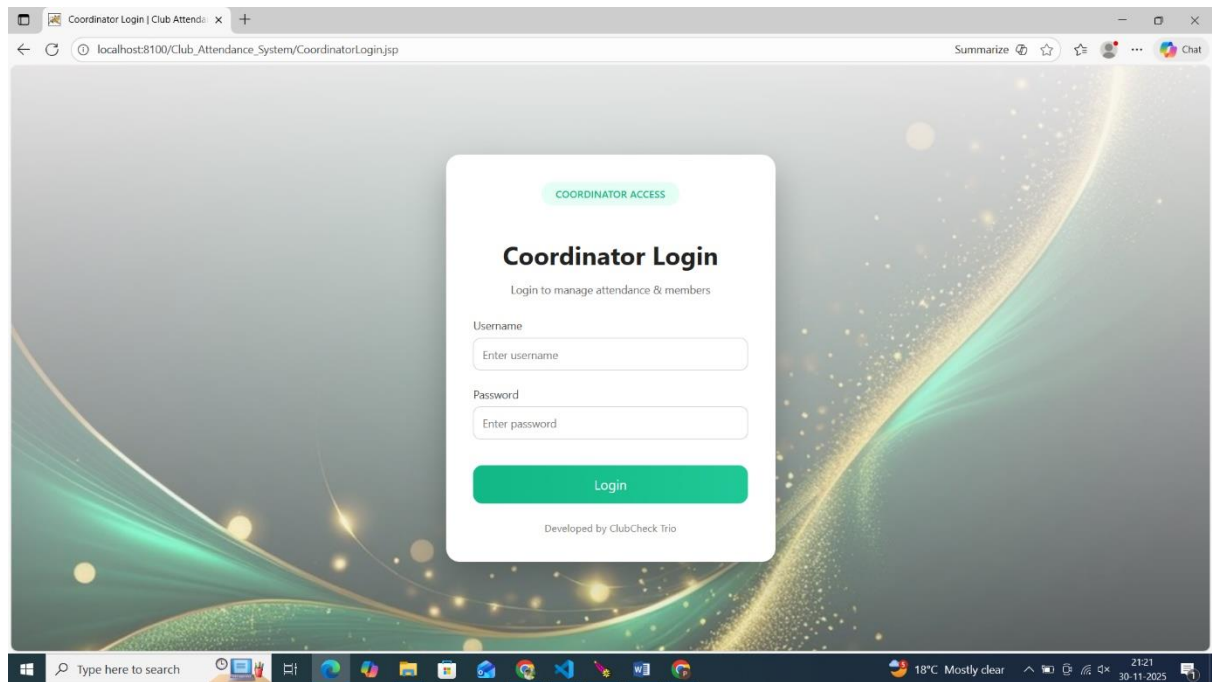


Image4 : Member Login Page

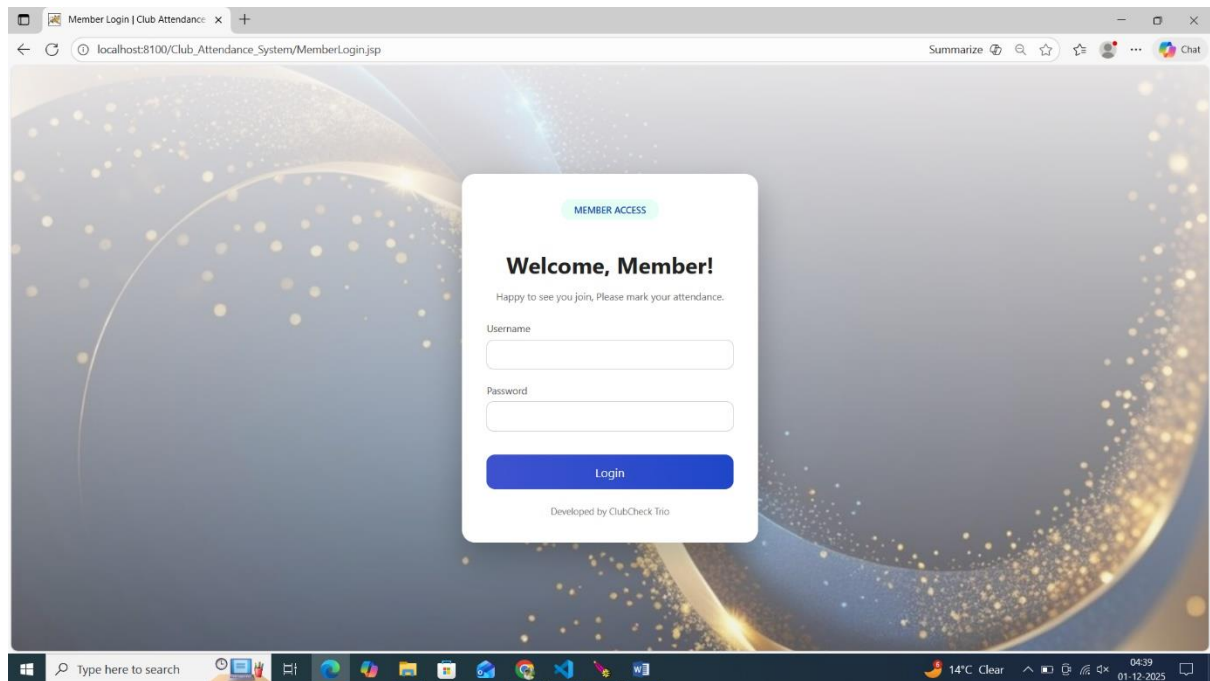


Image5 : Head Dashboard Page

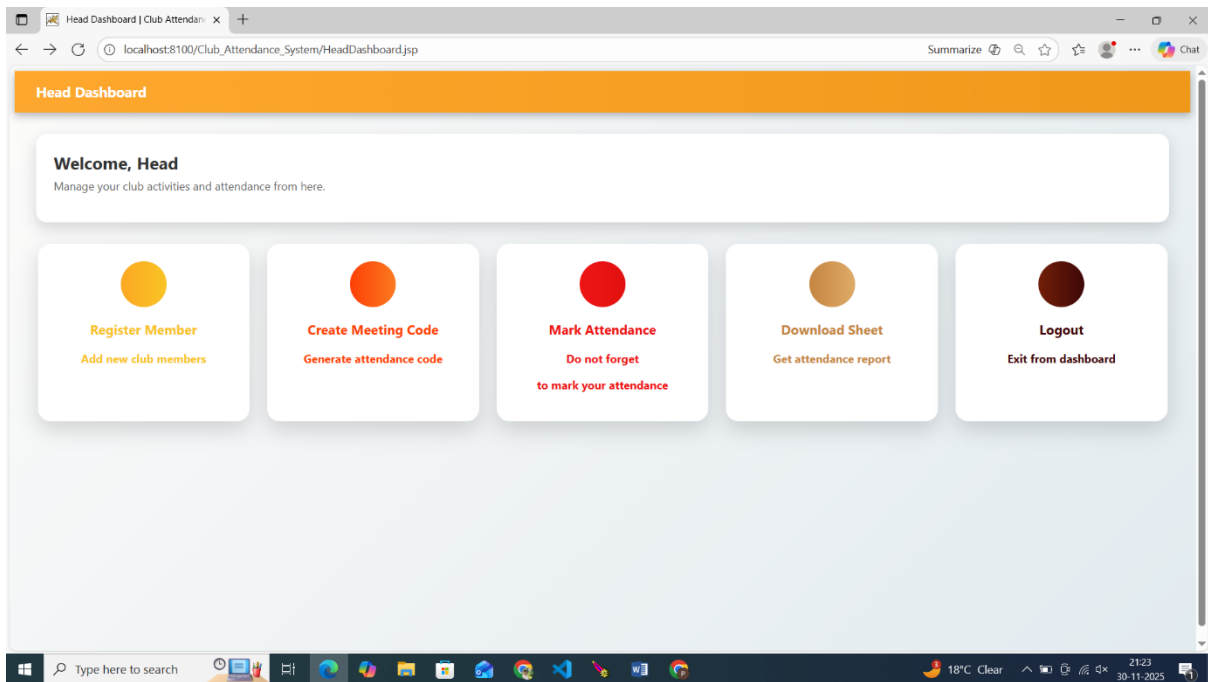


Image6 : Coordinator Dashboard Page

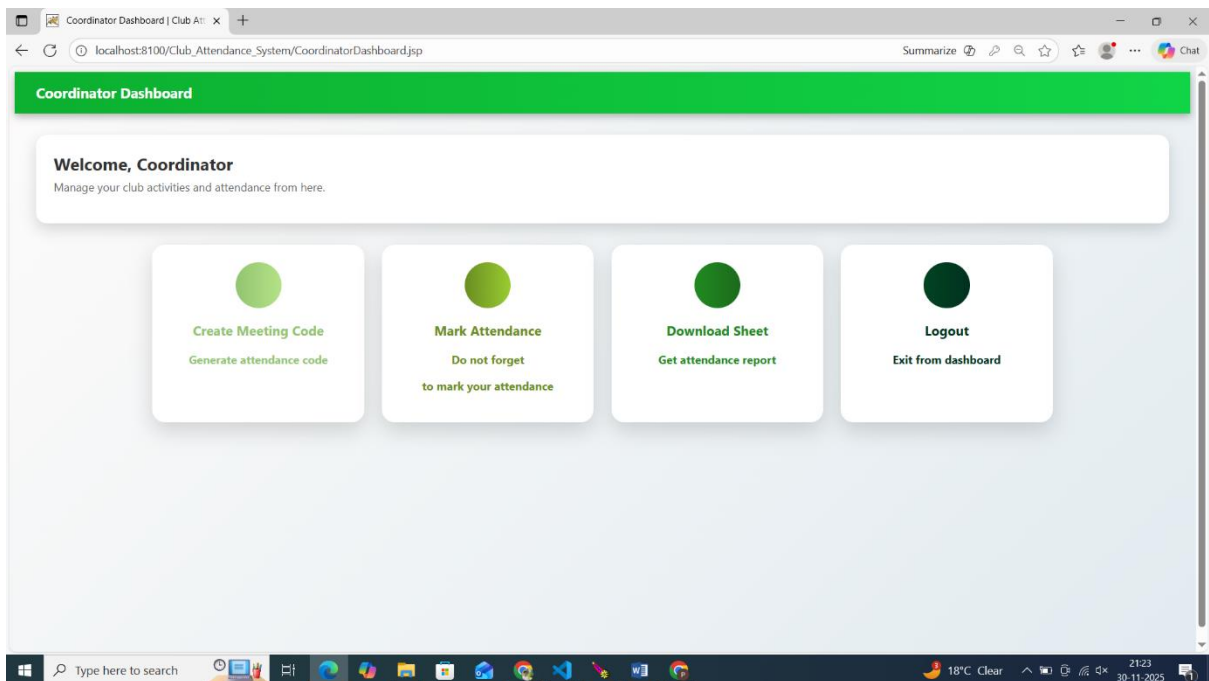


Image7 : Member Dashboard Page

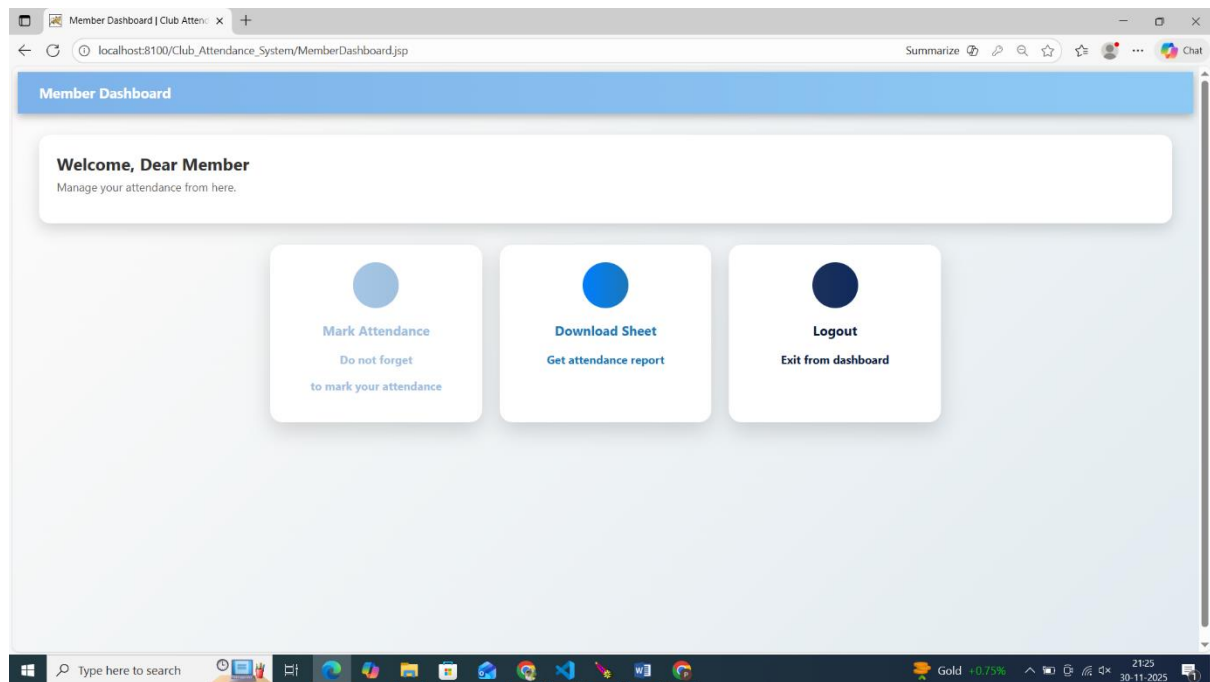


Image8 : Member Registration Page

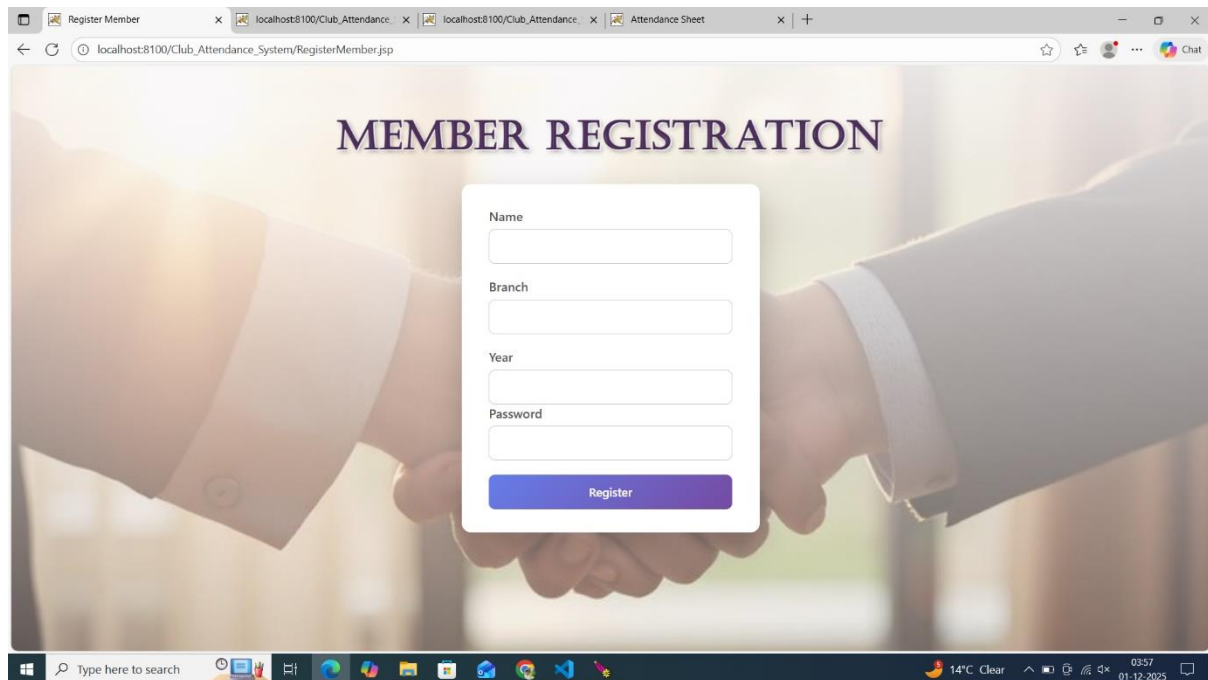


Image9 : Meeting Code Creation Page

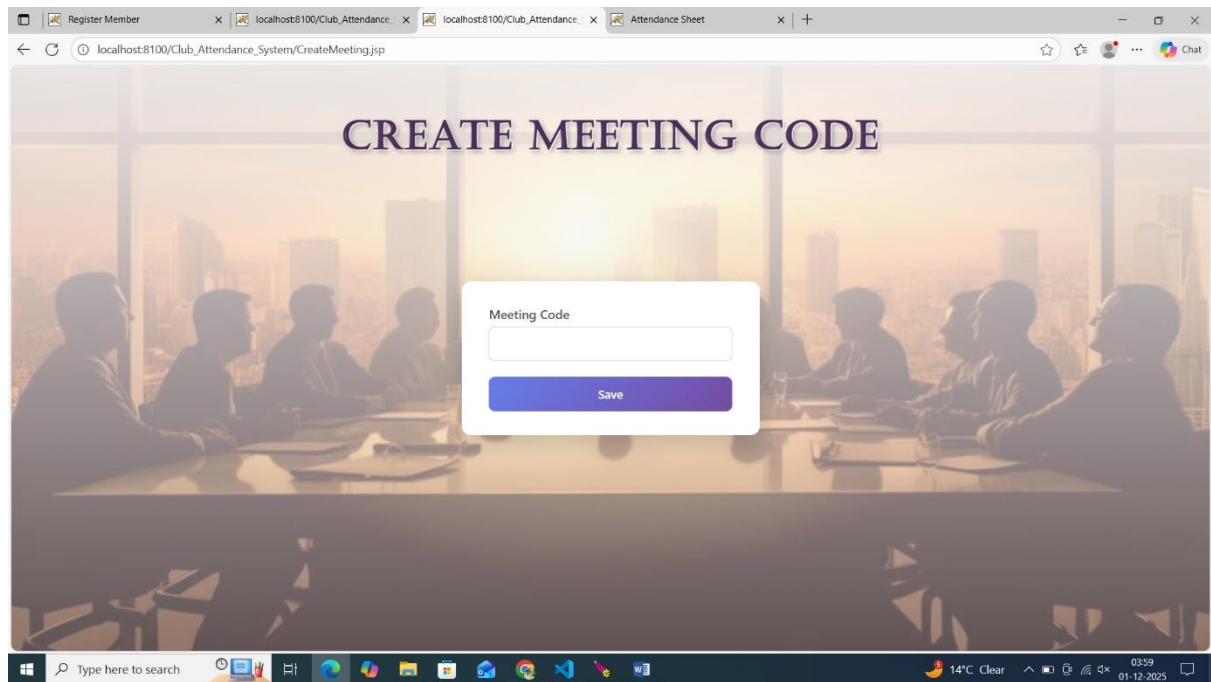


Image10 : Attendance Marking Page

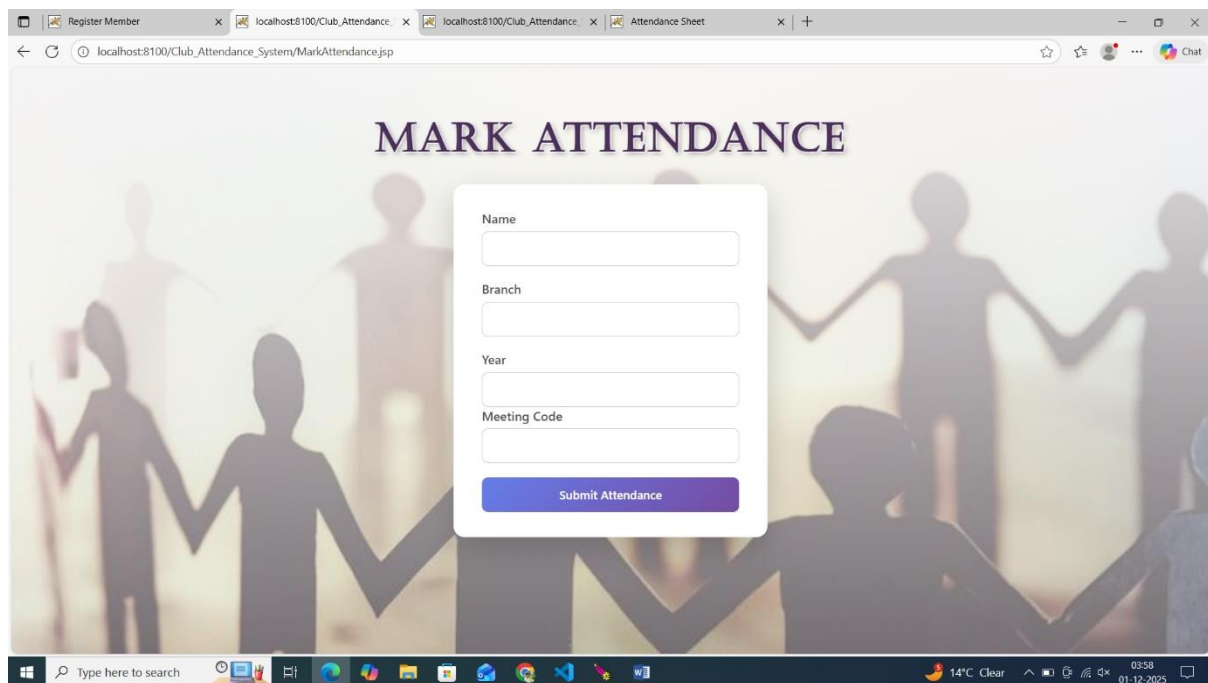


Image11 : Attendance Sheet View Page

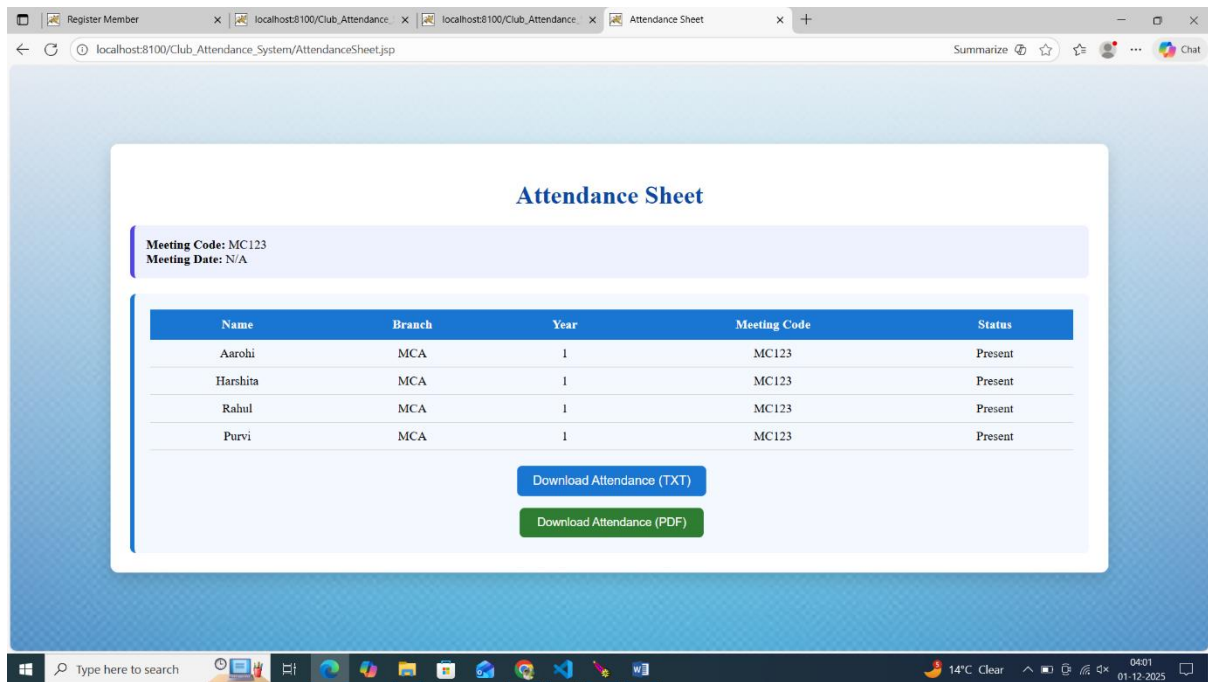


Image12 : Showing Denial of Login for Invalid Credentials

(Showing how system denies login upon entering wrong credentials for all three persons. Every login page is directed towards respective dashboard only if the login credentials are correct.)

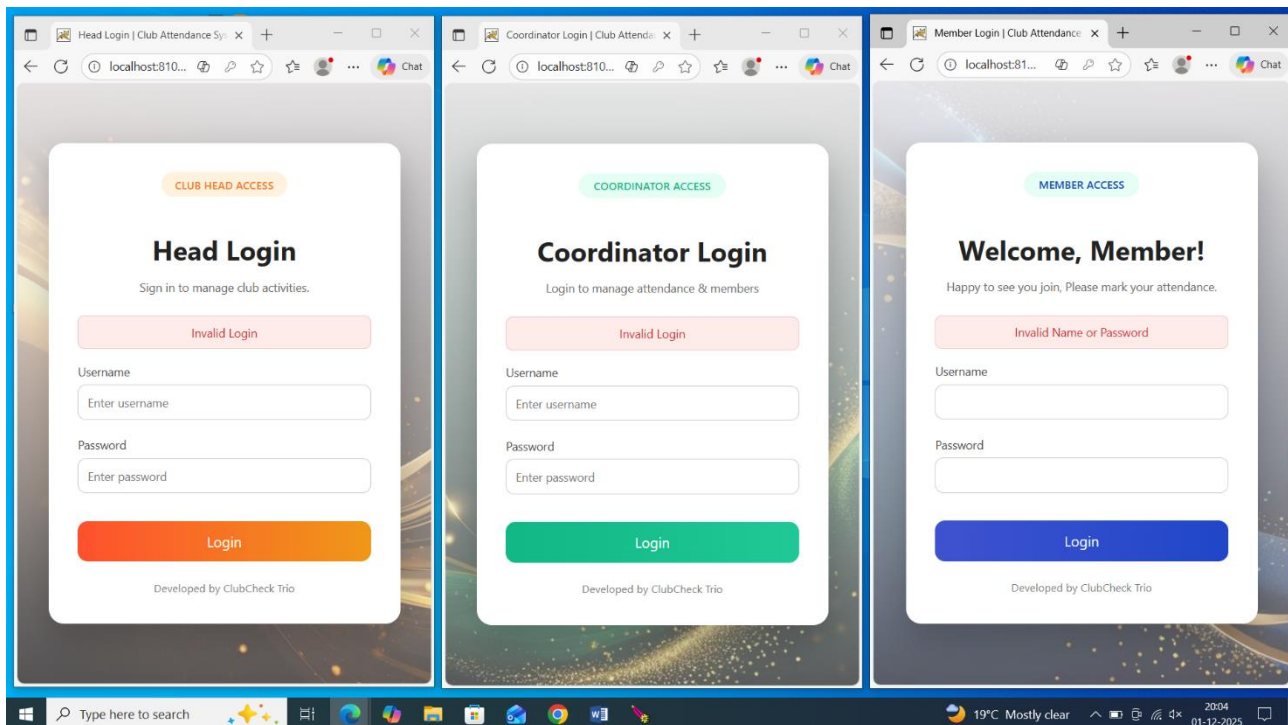
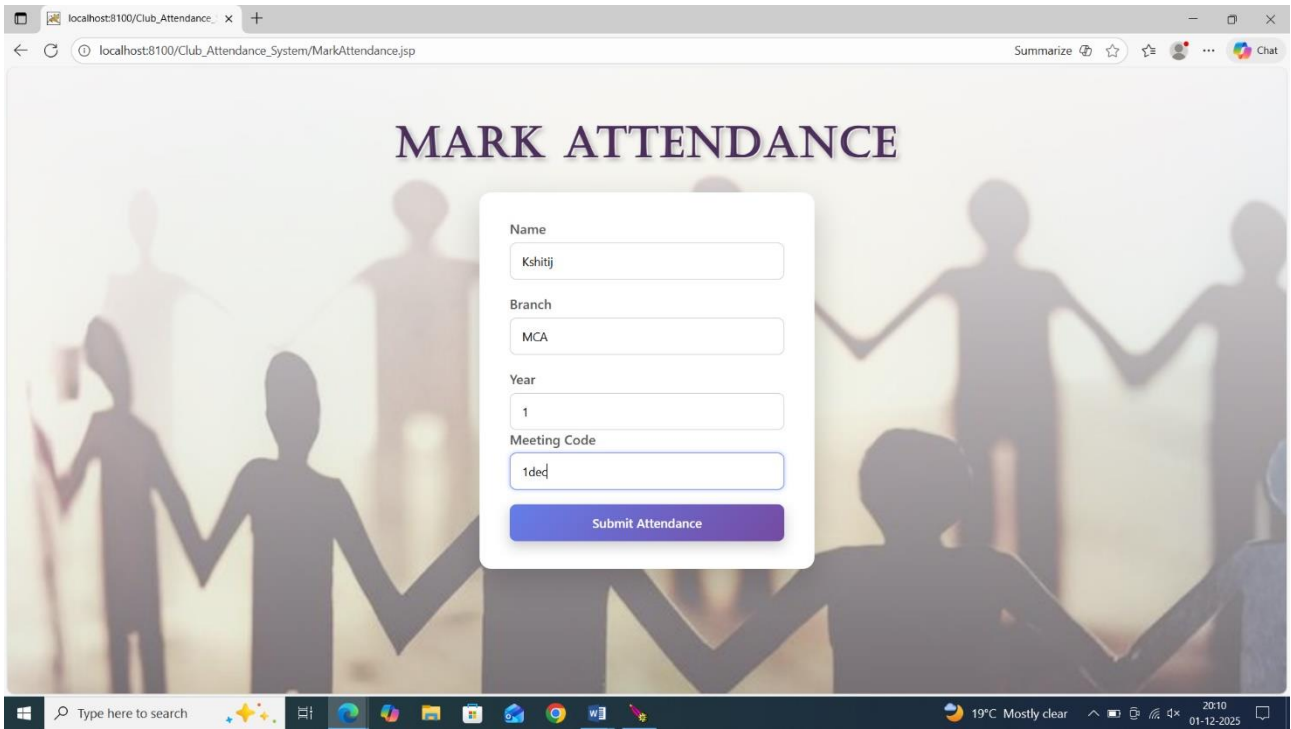


Image13 : Showcasing how member can mark attendance.

(Member can mark his/her attendance from there login by entering correct meeting code.)

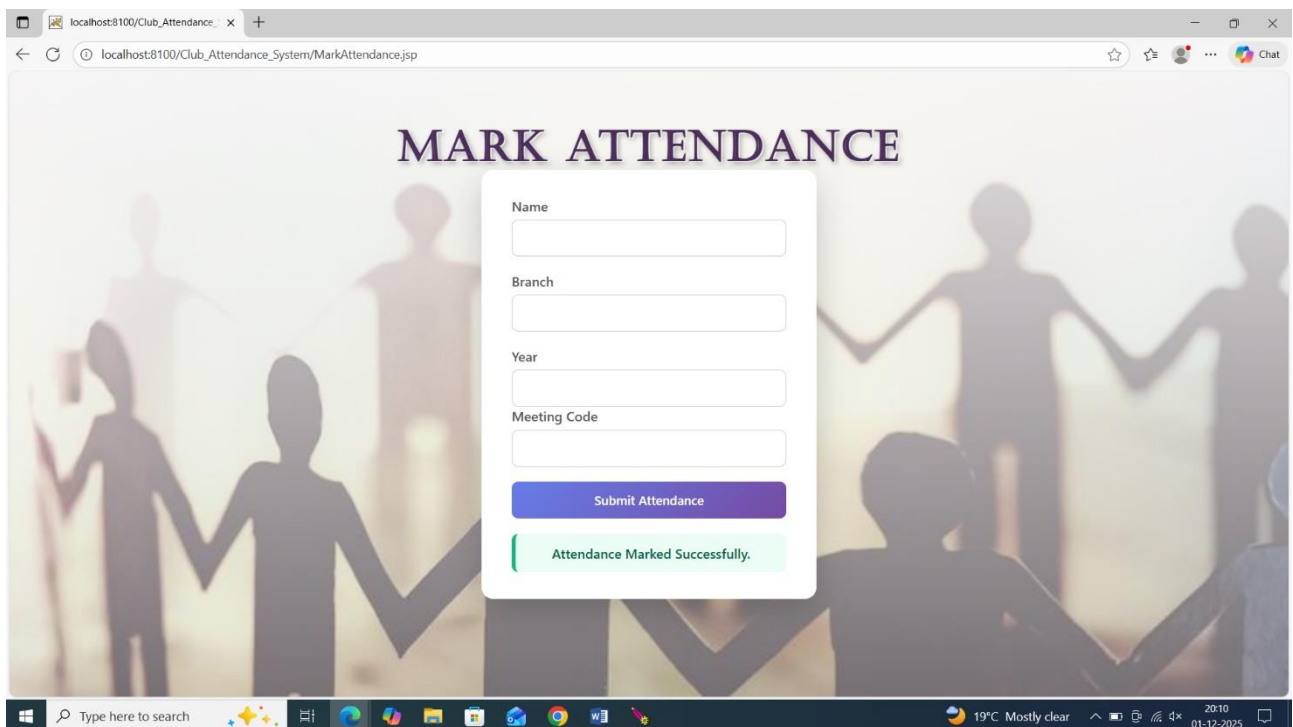


The screenshot shows a web browser window with the URL `localhost:8100/Club_Attendance_System/MarkAttendance.jsp`. The page title is "MARK ATTENDANCE". The form has the following fields:

- Name:
- Branch:
- Year:
- Meeting Code:
- Submit Attendance:

Image14 : Showing attendance is marked only for correct meeting code.

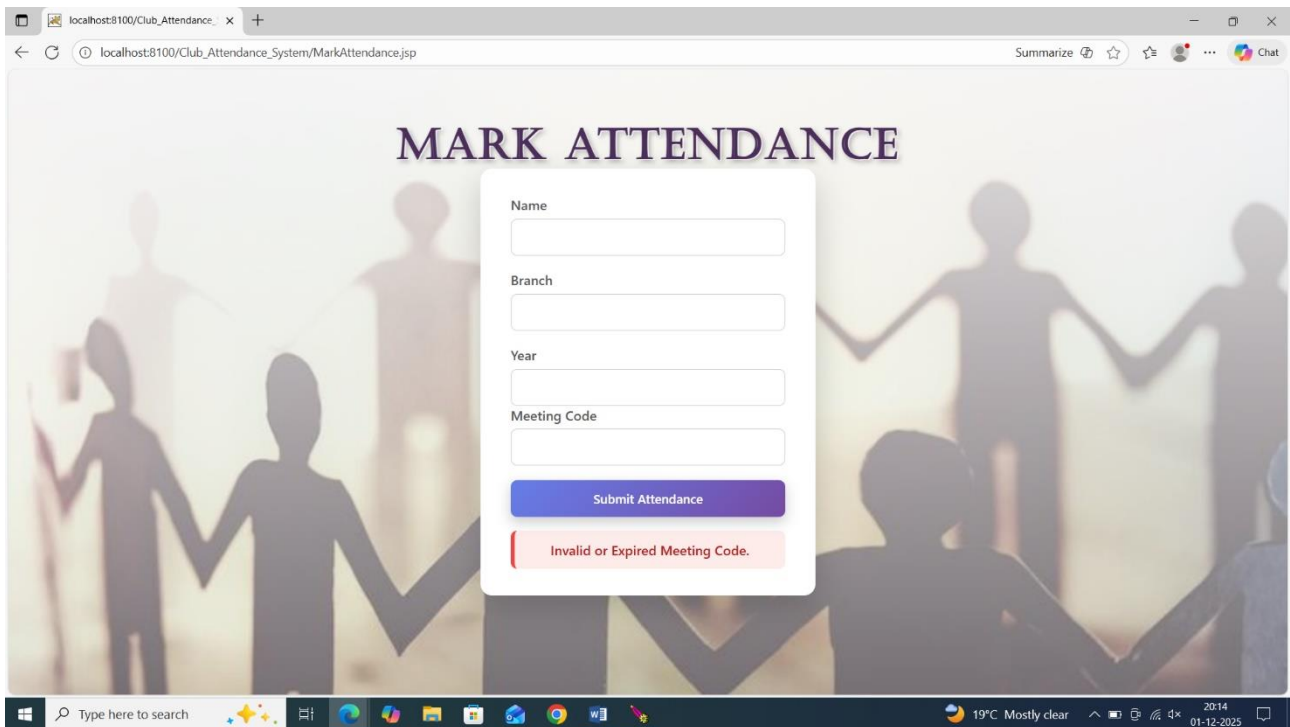
(Only if the meeting code entered by a member matches with most recent meeting code in a test file meetingcodes.txt, then member's attendance gets marked.)



The screenshot shows the same web browser window. The form fields are now empty. A green message box at the bottom of the form says "Attendance Marked Successfully." The "Submit Attendance" button is still present.

Image15 : Showing for incorrect meeting code, Attendance is not marked.

(If the most recent meeting code in a test file meetingcodes.txt, do not match with the one which is entered by member, then member's attendance does not get marked.)



RESULT AND TESTING

The system was tested to ensure correctness, security, and usability.

Result:

- The Club Attendance System was successfully developed and executed using JSP, file handling, and session management.
- The system correctly allows role-based login for Head, Coordinator, and Member.
- Members are able to log in using registered credentials stored in text files.
- Attendance is marked only when a valid meeting code is entered.
- The attendance sheet successfully displays:
 - Latest meeting code
 - Meeting date
 - List of present members
- Attendance reports can be downloaded in TXT and PDF format.
- Unauthorized access to dashboards is restricted using session validation.
- The system eliminates manual attendance errors and improves efficiency.

Security Testing

- Session prevents direct dashboard access without login.
- Role-based redirection works correctly.
- Invalid meeting code prevents fake attendance.

TEST CASE 1: Head Login

Input	Expected Output	Result
Correct username & password	Head Dashboard opens	Pass
Incorrect credentials	Error message displayed	Pass

TEST CASE 2: Member Registration

Input	Expected Output	Result
Valid member details	Member added to member.txt	Pass
Empty fields	Registration denied	Pass

TEST CASE 3: Meeting Code Generation

Input	Expected Output	Result
Create meeting	Code & date saved to file	Pass

TEST CASE 4: Member Login

Input	Expected Output	Result
Valid credentials	Member Dashboard appears	Pass
Invalid credentials	Error message shown	Pass

TEST CASE 5: Attendance Marking

Input	Expected Output	Result
Correct meeting code	Attendance marked Present	Pass
Wrong meeting code	Attendance rejected	Pass

TEST CASE 6: Attendance Sheet Display

Input	Expected Output	Result
View attendance	Attendance shown in table	Pass
No records	“No Attendance Found”	Pass

TEST CASE 7: Download Attendance

Action	Expected Output	Result
Download TXT	Text file saved	Pass
Download PDF	Print → Save as PDF	Pass

Final Result Conclusion

The Club Attendance System meets all the project objectives and performs reliably under different test scenarios. The system is user-friendly, secure, and suitable for real-time club attendance management.

CONCLUSION

The **Club Attendance System** was successfully designed and implemented using **Java Server Pages (JSP)**, file handling, and session management. The system provides a secure and efficient method for managing attendance digitally, replacing traditional manual processes.

The project achieves its objective of **role-based access control** by allowing only authorized users such as the Head, Coordinator, and Members to perform specific operations. Attendance marking using **meeting codes** ensures authenticity and prevents unauthorized entries. The system also provides features such as attendance viewing, downloading reports, and session-based security.

Overall, the project demonstrates a practical implementation of web application concepts including authentication, file handling, and user interface design, making it suitable for academic usage and small-scale organizational environments.

FUTURE SCOPE

Although the system functions effectively, it can be further enhanced with additional features to improve scalability, security, and usability.

1. Database Integration

- Replace text files with a relational database (MySQL / Oracle) for better performance and data security.

2. Password Encryption

- Implement hashing techniques such as SHA-256 or BCrypt to securely store passwords.

3. Email / OTP Verification

- Send meeting codes or login verification via email or SMS for enhanced authentication.

4. Biometric or QR Code Attendance

- Implement QR code or biometric-based attendance for faster and more secure entries.

5. Mobile Application Support

- Extend the system into a mobile application using Android or Flutter.

6. Analytics and Reports

- Generate graphical reports such as attendance percentage, charts, and summaries

7. Admin Panel Enhancements

- Allow editing or deleting of member records and attendance data

8. Cloud Deployment

- Host the application on cloud platforms for remote access and scalability.

REFERENCES

The project was developed using official Java and JSP documentation, online tutorials, and standard web development resources.

Guide's Lecture Notes

Notes and classroom instruction provided by project guide during project development.

Additional Online Resources

Various online forums and documentation for problem solving.

Oracle Corporation

Java Platform, Standard Edition Documentation

<https://docs.oracle.com/javase/>

Apache Software Foundation

Apache Tomcat Documentation

<https://tomcat.apache.org/documentation.html>

GeeksforGeeks

Java File Handling and JSP Concepts

<https://www.geeksforgeeks.org>